The Current `Dynamics` Class

1 Purpose

The current `Dynamics` class is the collision-response kernel for the clean rebuild. It is intentionally separated from `Base`. The `Base` class owns the simulation loop, particle storage, contact detection, double-buffered position movement, and reporting. The `Dynamics` class owns only the math that converts detected contacts into momentum changes.

This separation is important because the dynamics code is the part most likely to be moved later into C++ or GLSL. The current design is therefore written with a simple kernel-like rule:

one worker owns one particle.

During collision processing, the worker for particle i may read selected state from another particle j, but it writes only to particle i.

2 Particle Data Contract

Each particle is currently stored as a Python dictionary. The dynamics class expects the following fields to exist before collision processing begins.

2.1 Position

Position is double-buffered:

$$\texttt{location} = \{\texttt{use}, \texttt{x1}, \texttt{y1}, \texttt{x2}, \texttt{y2}\}.$$

If

$$\texttt{use} = 1,$$

then the active position is

$$\mathbf{x} = (x_1, y_1).$$

If

$$\texttt{use} = 2,$$

then the active position is

$$\mathbf{x} = (x_2, y_2).$$

The helper `current_location()` reads this active position.

2.2 Particle Properties

The dynamics code reads:

$$r_i = \texttt{radius}, \qquad m_i = \texttt{mass}, \qquad \mathbf{v}_i = (v_{x,i}, v_{y,i}).$$

The current overlap-area formula assumes equal-radius disks:

$$r_i = r_j = r.$$

If unequal radii are passed to `circle_overlap_area()`, the code raises an error.

2.3 Contact List

`Base.detect_contacts()` fills each particle's contact storage before `Dynamics.process_collisions()` runs. For particle i, the relevant fields are:

$$\texttt{contact\_count}_i, \qquad \texttt{contacts}_i.$$

`contacts` is a fixed-length array of particle indices. Only the first `contact_count` entries are valid.

3 Memory Ownership Rule

The dynamics pass is particle-owned. When processing particle i, the code loops through i's own contacts:

$$j \in \texttt{contacts}_i.$$

The code may read particle j's current location and radius to calculate geometry. It does not write to particle j. This is different from a traditional pair solver that updates both particles in one pair operation.

The current update has the form:

$$\Delta\mathbf{p}_i = \sum_{j \in C_i} \Delta\mathbf{p}_{ij}.$$

The corresponding update for particle j is produced only when particle j's own worker processes its own contact list.

4 Contact Geometry

For a pair i, j, the active center positions are:

$$\mathbf{x}_i = (x_i, y_i), \qquad \mathbf{x}_j = (x_j, y_j).$$

The separation vector from particle i to particle j is:

$$\mathbf{d}_{ij} = \mathbf{x}_j - \mathbf{x}_i.$$

The center distance is:

$$d = \|\mathbf{d}_{ij}\|.$$

Contact exists when:

$$d < r_i + r_j.$$

If there is no overlap, `particle_contact()` returns `None`.

4.1 Contact Normal

When $d > 0$, the unit normal from particle i toward particle j is:

$$\mathbf{n}_{ij} = \frac{\mathbf{x}_j - \mathbf{x}_i}{d} = (n_x, n_y).$$

If the centers are effectively identical,

$$d \leq 10^{-12},$$

the code uses the deterministic fallback:

$$\mathbf{n}_{ij} = (1, 0).$$

This avoids division by zero.

5 Penetration Limit Used For Area

The raw overlap depth is:

$$\delta_{\text{raw}} = r_i + r_j - d.$$

The current code caps the depth used for area calculation:

$$\delta = \min\left(r_i + r_j - d, \min(r_i, r_j)\right).$$

Under the equal-radius assumption, this means:

$$\delta \leq r.$$

This matches the current development assumption that particles do not move past the case where the leading edge reaches the other particle center.

The distance sent to the overlap-area formula is:

$$d_A = r_i + r_j - \delta.$$

6 Equal-Radius Overlap Area

For equal-radius particles,

$$r_i = r_j = r,$$

the circular lens area is:

$$A(d_A) = 2r^2 \cos^{-1}\left(\frac{d_A}{2r}\right) - \frac{d_A}{2}\sqrt{4r^2 - d_A^2}.$$

The implementation clamps the distance into the valid range:

$$d_c = \min(\max(d_A, 0), 2r).$$

The actual computed area is:

$$A(d_c) = 2r^2 \cos^{-1}\left(\frac{d_c}{2r}\right) - \frac{d_c}{2}\sqrt{\max(0, 4r^2 - d_c^2)}.$$

The $\max(0, \cdot)$ term protects the square root from small floating-point roundoff errors.

7 Overlap Momentum

The current model follows the minimal legacy Mom-style idea:

$$\text{overlap area} \rightarrow \text{momentum amount}.$$

It does not treat overlap as a force integrated over the substep. There is no multiplication by Δt inside `overlap_momentum()`.

For a contact between particle i and particle j, the scalar momentum amount is:

$$J_{ij} = k_i A_{ij} w(d).$$

Here:

$$k_i = \texttt{momentum\_per\_area}$$

and:

$$A_{ij} = \text{overlap area}.$$

The coefficient k_i is owned by particle i. If particle i has a `momentum_per_area` field, that value is used. Otherwise the class default is used. The code also keeps the legacy fallback name `repulsion_force_per_area`.

8 Inverse-Square Weight

The distance weight is:

$$w(d) = \frac{1}{\max(d^2, s^2)}.$$

where

$$s = \texttt{inverse\_square\_softening}.$$

The softening value prevents singular behavior when two centers are very close:

$$s \geq 10^{-12}.$$

9 Momentum Direction

The contact normal $\mathbf{n}_{ij}$ points from particle i toward particle j. The collision response for particle i points away from particle j, so the update for this contact is:

$$\Delta \mathbf{p}_{ij} = -J_{ij}\mathbf{n}_{ij}.$$

Component-wise:

$$\Delta p_{x,ij} = -n_x J_{ij}, \qquad \Delta p_{y,ij} = -n_y J_{ij}.$$

10 Accumulating Contacts

For a particle with contact set C_i, the dynamics class accumulates:

$$P_{x,i} = \sum_{j \in C_i} -n_{x,ij} J_{ij},$$

$$P_{y,i} = \sum_{j \in C_i} -n_{y,ij} J_{ij}.$$

It also tracks the scalar sum:

$$P_{\text{sum},i} = \sum_{j \in C_i} J_{ij}.$$

These values are written back onto particle i as:

$$\begin{aligned} \texttt{overlap\_momentum\_x} &= P_{x,i}, \\ \texttt{overlap\_momentum\_y} &= P_{y,i}, \\ \texttt{overlap\_momentum} &= P_{\text{sum},i}. \end{aligned}$$

The same vector is currently also stored as internal momentum:

$$\begin{aligned} \texttt{internal\_momentum\_x} &= P_{x,i}, \\ \texttt{internal\_momentum\_y} &= P_{y,i}, \\ \texttt{internal\_momentum} &= \sqrt{P_{x,i}^2 + P_{y,i}^2}. \end{aligned}$$

11 Velocity Update

The method `apply_overlap_momentum()` converts the stored momentum vector into a velocity change:

$$\Delta \mathbf{v}_i = \frac{\Delta \mathbf{p}_i}{m_i}.$$

Component-wise:

$$v_{x,i}^{\text{new}} = v_{x,i} + \frac{\texttt{overlap\_momentum\_x}}{m_i},$$

$$v_{y,i}^{\text{new}} = v_{y,i} + \frac{\texttt{overlap\_momentum\_y}}{m_i}.$$

The position update for the current substep happens in `Base` before collision processing. Therefore this velocity change affects the next movement step.

12 Current Step Order

At the current rebuild stage, the relevant order is:

1. `Base` predicts the next position using current velocity.
2. `Base.move()` flips the active location buffer.
3. `Base.detect_contacts()` fills each particle's contact list.
4. `Dynamics.process_collisions()` computes overlap momentum.
5. `Dynamics.apply_overlap_momentum()` updates velocity.
6. `Base.record_step_report()` stores per-particle values.

13 What This Class Does Not Do

The current `Dynamics` class does not include bonding, field dynamics, long-term internal momentum release, classical restitution coefficient e, or pair-store bookkeeping. It is only the minimal overlap-momentum collision kernel.

It also does not update both particles inside a single pair operation. That is intentional for the current particle-owned memory model.

14 Future Porting Notes

The most direct C++ or GLSL version would give each particle one worker. That worker would:

1. read particle i's contact count and contact array,
2. loop through the valid contact slots,
3. read particle j's current position and radius,

4. compute A_{ij}, J_{ij}, and $-J_{ij}\mathbf{n}_{ij}$,

5. write only particle i's momentum outputs and updated velocity.

The current Python class is shaped to keep that translation straightforward.